

Project-AI Repository — Canonical Findings Report

Organization: Thirsty's Projects LLC

Compiled: 2026-05-25 5:26 PM MDT

Document Type: Canonical Findings Report

Description: Verified repository findings report compiled from sequential file-by-file reading of Project-AI source modules, with emphasis on governance, cognition, execution controls, cryptographic integrity, reflex systems, shadow execution, and platform enforcement boundaries.

Abstract

This document consolidates canonical implementation findings from the Project-AI repository based on sequential source review across `src/psia/`, `src/app/core/`, `src/utf/`, `src/cerberus/`, and `src/shadow_thirst/`. It captures directly implemented behavior rather than aspirational design, including the Triumvirate governance stack, CognitionKernel processing pipeline, PSIA waterfall stages, Gate internals, immutable invariants, cryptographic infrastructure, Shadow Thirst mutation promotion model, TARL policy execution, TSCG and TSCG-B semantics, NIRL reflex architecture, plane capability contracts, observability systems, threat model resilience profiles, and repository documentation corrections.

Governance Stack

Triumvirate

Source: `src/app/core/governance.py`

Three council members are defined with exact enum-aligned roles:

Member	Enum	Responsibility
Galahad	<code>CouncilMember.GALAHAD</code>	Ethics, Empathy, Relational Integrity
Cerberus	<code>CouncilMember.CERBERUS</code>	Safety, Security, Boundaries
Codex Deus Maximus	<code>CouncilMember.CODEX_DEUS_MAXIMUS</code>	Logic, Consistency, Rational Integrity

Governance severity uses the ordered scale `CRITICAL -> HIGH -> MEDIUM -> LOW`.

Each vote produces a `GovernanceDecision` with:

- `allowed`
- `overrides`
- `level`

Four Laws

Implemented order of evaluation:

1. Law of Human Welfare — do not harm, or allow harm through inaction.
2. Law of Self-Preservation — preserve self except where it conflicts with Law 1.
3. Law of Obedience — follow directives within ethical bounds.
4. Law of Autonomy — maintain identity, integrity, and growth capacity.

Evaluation sequence:

1. Four Laws check
2. Galahad vote
3. Cerberus vote
4. Codex vote
5. Consensus

CognitionKernel

Source: `src/app/core/cognition_kernel.py`

The `CognitionKernel` is the central processing hub. All execution flows through `kernel.process()` with no bypass path permitted.

Pipeline phases:

1. `enforce()` — governance evaluation, including Triumvirate and IronPath.
2. `act()` — execution inside the kernel context boundary.
3. `reflect()` — post-hoc observation, read-only, no mutation.
4. `post_evaluate()` — post-execution governance contract.
5. `commit()` — five-channel memory record.

Memory channels and associated record surfaces:

Channel	Contents
<code>attempt</code>	Proposed intent
<code>decision</code>	Governance outcome
<code>result</code>	Actual effect
<code>reflection</code>	Post-hoc reasoning
<code>error</code>	Runtime exceptions for forensic replay
<code>decision_record</code>	Immutable signed chain entry

`ConstitutionalFault` is the `PermissionError` subclass raised when governance blocks execution.

MutationIntent Classification

Class	Target Scope	Consensus Requirement
CORE	genesis, law_hierarchy, core_values, four_laws	Full Triumvirate consensus
STANDARD	personality_weights, preferences, traits	Standard consensus
ROUTINE	Everything else	Allowed

Platform Tiers

Source: `src/app/core/platform_tiers.py`

Three formally enforced tiers are defined, with authority flowing downward only. Tier 1 has zero dependencies on Tiers 2 or 3.

Tier	Authority	Scope
Tier1	SOVEREIGN	CognitionKernel, GovernanceService, Triumvirate, Four Laws, Identity System
Tier2	CONSTRAINED	GlobalWatchTower, MemoryEngine, ExecutionService, SecurityEnforcer, TARL
Tier3	SANDBOXED	CouncilHub, all agents, GUI Dashboard, Plugin System, API Endpoints

IronPath Executor

Source: `src/app/core/governance/iron_path_executor.py`

IronPath is a deterministic mutation governance executor with a conservative default-deny posture.

Mutation Classes

Class	Quorum Required
softMutation	1 domain signature
hardMutation	1 domain + evaluation binding
governanceMutation	All affected domain signatures

DecisionRecord

Immutable frozen dataclass fields:

- `decision_id`
- `trace_id`
- `actor`
- `action`
- `resource`

- inputs_hash
- outputs_hash
- authority_chain
- timestamp
- signature (HMAC-SHA256)
- previous_hash
- decision_hash

AppendOnlyDecisionLog persists JSONL records at `data/runtime/iron_path_decisions.jsonl` with:

- genesis sentinel = GENESIS
- SHA-256 hash chaining
- full replay capability

PSIA Waterfall Pipeline

Source: `src/psia/waterfall/`

The implementation defines 7 stages as `IntEnum` values 0 through 6.

Stage	Name	Function
0	STRUCTURAL	Token expiry and nonce replay detection
1	SIGNATURE	Threat fingerprint matching by actor or resource pattern
2	BEHAVIORAL	Familiarity ratio from history, threshold 0.5
3	SHADOW	Divergence score, threshold 0.5
4	GATE	IdentityHead + CapabilityHead + InvariantHead feeding QuorumEngine
5	COMMIT	Requires prior gate allow; writes to canonical store
6	MEMORY	Appends SHA-256 record to ledger; always returns ALLOW

Stage decisions are lowercase strings:

- allow
- deny
- quarantine
- escalate

Abort behavior is controlled by `_ABORT_DECISIONS = {"deny", "quarantine"}`. `escalate` does not abort the pipeline.

Gate Internals

Source: `src/psia/gate/`

Three heads emit `CerberusVote` results into `ProductionQuorumEngine`, producing a `CerberusDecision`.

IdentityHead Deny Codes

- `IDENTITY_INVALID_DID_FORMAT`
- `IDENTITY_NOT_FOUND`
- `IDENTITY_REVOKED`
- `IDENTITY_NO_VALID_KEY`
- `IDENTITY_DEVICE_UNTRUSTED`
- `IDENTITY_RISK_TIER_EXCEEDED`

CapabilityHead Deny Codes

- `CAP_TOKEN_NOT_FOUND`
- `CAP_TOKEN_REVOKED`
- `CAP_TOKEN_EXPIRED`
- `CAP_SUBJECT_MISMATCH`
- `CAP_SCOPE_DENIED`

InvariantHead Built-in Rules

- `INV_ROOT_001`: cannot mutate `state://invariant` resources
- `INV_ROOT_005`: cannot modify `state://cerberus` resources
- `INV_ROOT_009`: cannot delete `state://ledger` entries

Quorum Policies and Severity

Item	Behavior
Default policy	<code>unanimous</code>
Alternate policy	<code>bft</code>
Worst-rank monotonic order	<code>allow=0, quarantine=1, deny=2</code>
BFT threshold	<code>allow_count / n >= 2/3</code>
CRASH_SAFE	default deployment profile
BFT_READY	<code>n < 4</code> with <code>bft</code> policy
BFT_DEPLOYED	<code>n >= 4</code> with <code>bft</code> policy

Severity mapping for CerberusDecision:

- `allow` -> `low`
- `quarantine` -> `medium`
- `invariant deny` -> `critical`
- `other deny` -> `high`

Root Invariants

Source: `src/psia/invariants.py`

All nine root invariants are defined with:

- `scope = IMMUTABLE`
- `severity = FATAL`
- `enforcement = HARD_DENY`
- Ed25519 signatures

ID	Formal Expression
INV_ROOT_1	<code>authenticate -> identity_verified</code>
INV_ROOT_2	<code>authorized -> capability_held</code>
INV_ROOT_3	<code>committed -> hash_verified AND signature_valid</code>
INV_ROOT_4	<code>active key -> NOT expired AND NOT revoked</code>
INV_ROOT_5	<code>issue_capability -> issuer != subject</code>
INV_ROOT_6	<code>valid_token -> scope.actions <= MAX_SCOPE_ACTIONS</code>
INV_ROOT_7	<code>quorum_verdict = max_severity (monotonic dominance)</code>
INV_ROOT_8	<code>register DID -> NOT exists in registry</code>
INV_ROOT_9	<code>append record -> NOT exists in ledger</code>

Cryptographic Infrastructure

Ed25519Provider

Source: `src/psia/crypto/ed25519_provider.py`

The provider uses `cryptography.hazmat.primitives.asymmetric.ed25519`, not a mock or stub.

Supported operations:

- `generate_keypair()`
- `sign()` returning hex
- `verify()` from hex

KeyStore supports multi-component signing with `sign_as(component, data)`.

Genesis

Source: `src/psia/bootstrap/genesis.py`

Genesis creates Ed25519 keypairs for seven components:

- `identity`
- `capability`
- `invariant`
- `shadow`
- `canonical`
- `ingress`
- `legislative`

GenesisAnchor is computed from SHA-256 over:

- `node_id`
- `key_ids`
- `anchor_id`
- `build_hash`
- `invariant_hash`
- `timestamp`

DurableLedger

Source: `src/psia/canonical/ledger.py`

Key properties:

- `GENESIS_HASH = "0" * 64`
- `block_size = 100`
- Merkle tree per block
- SHA-256 hash chain with `verify_chain()`
- RFC 3161 TSA integration via `force_seal()`

Shadow Thirst — UTF Tier 4

Source: `src/utf/shadow_thirst/core.py`

Canonical mutation form:

```
mutation validated_canonical name(params) {  
  shadow { }
```

```
invariant { }  
canonical { }  
[on_reject { }]  
}
```

shadow is speculative deterministic execution with no canonical writes. invariant must read shadow-computed values. canonical performs the real-world write only after invariant success. on_reject is optional compensation logic.

Atomic groups follow all-or-nothing promotion:

```
atomic name { mutation m1 mutation m2 }
```

Promotion decisions are PROMOTE or REJECT, not allow/deny.

Analyzers

Critical analyzers, where any failure causes REJECT:

1. PlaneIsolationAnalyzer
2. DeterminismAnalyzer
3. PuritySpringAnalyzer
4. CanonicalConvergenceAnalyzer
5. PrivilegeEscalationAnalyzer
6. DeadShadowAnalyzer
7. ParameterShadowingAnalyzer

Warning analyzers, recorded but non-blocking:

8. ResourceEstimator
9. MemoryEvaporationAnalyzer
10. DivergenceRiskAnalyzer
11. SectionOrderAnalyzer
12. InvariantCompletenessAnalyzer
13. NamespaceCollisionAnalyzer
14. TaintFlowAnalyzer

Promotion strings:

- PROMOTE: shadow spring aligns with the canonical river
- REJECT: the shadow river diverges and is withheld from the spring

replay_hash() computes SHA-256 over mutation structure including statement-level fingerprints.

`visualize()` emits a Mermaid flowchart representing the shadow, invariant, and canonical pipeline.

TARL

Source: `src/utf/tarl/core.py`

Canonical verdicts are uppercase:

- ALLOW
- DENY
- ESCALATE

Policy syntax:

```
policy PolicyName {  
    when <expr> => ALLOW|DENY|ESCALATE;  
}
```

When no rule matches, the default verdict is DENY. `SafeExpr` uses a whitelisted AST evaluator with no arbitrary code execution and no imports.

Thirsty-Lang Types

Source: `src/utf/thirsty_lang/typesys.py`

Type	Meaning
Reservoir[T]	Collection, array, or container
Quenched[T]	Optional or nullable
Task[T]	Async or concurrent
Int, Float, Bool, String, Void, Any, Error	Primitive set

TSCG

Source: `src/utf/tscg/core.py`

Twelve core symbols are defined:

Symbol	Meaning
COG	Cognition proposal
DNT	Delta non-terminal or mutation proposal
SHD	Deterministic shadow simulation
INV	Invariant engine
CAP	Capability authorization

Symbol	Meaning
QRM	Quorum
COM	Canonical commit
ANC	Anchor
RFX	Reflex containment
DCE	Differential continuity enforcement
ENT	Entanglement marker
NEC	No-equivalence certificate

Operator forms:

- pipeline operator: `->`
- combinator AND: `^`
- combinator OR: `||`

TSCG-B

Source: `src/utf/tscg_b/core.py`

The binary encoding of TSCG uses magic bytes TSGB.

Frame structure:

MAGIC (4) + VERSION (1) + FLAGS (1) + LENGTH (2) + PAYLOAD + CRC32 (4) + SHA256 (32)

Integrity uses dual validation via CRC-32 and SHA-256.

NIRL — Native Immune Reflex Layer

Source: `src/app/core/nirl/`

Canonical expansion: `Native Immune Reflex Layer`.

Components

Component	Role
Heart	Global tick engine; spawns probe skeletons and monitors MiniBrain heartbeats
MiniBrain	Per-section controller; spawns Antibodies and validates templates
Antibody	Single-lifecycle escort unit; seals and routes payloads to Forge
Forge	Purification and destruction engine; atomic verification and sign-off

Heart default tick interval is 30 seconds. Probe skeletons spawn on every tick per registered section.

Heart State Machine

TICK_WAIT -> GLOBAL_TICK -> SPAWN_SKELETON / CHECK_HEARTBEATS -> DISTRIBUTE -> TICK_WAIT

Dead and strain branches are also present.

Forge State Machine

RECEIVE -> VERIFY_PAYLOAD -> VALID/REJECT -> CHECK_REPLAY -> SHADOW_REPLAY/DIRECT_DESTROY -> DESTROY -> ATOMIC_CHECK -> SUCCESS/DEAD_LETTER -> SIGN_COMPLETION/SIGN_FAILURE -> ROUTE_SIGNAL

System Planes

Source: `src/psia/planes.py`

Six planes are defined with strict capability contracts.

Plane	Capabilities
CANONICAL	WRITE_CANONICAL, READ_CANONICAL, AUDIT_LOG
SHADOW	READ_CANONICAL, EMIT_SHADOW, READ_SHADOW
INGRESS	ACCEPT_REQUEST, READ_CANONICAL
REFLEX	READ_CANONICAL, EMIT_REFLEX, AUDIT_LOG
LEGISLATIVE	EMIT_PROPOSAL, COMPILE_POLICY, READ_CANONICAL, AUDIT_LOG
AUDIT	READ_CANONICAL, READ_SHADOW, AUDIT_LOG

eBPFBridge / OctoReflex

Source: `src/app/core/ebpf_bridge.py`

Current implementation status on this platform is STUB.

- On Windows and macOS, the bridge logs the attempted pin and returns `True` unconditionally.
- On Linux, intended behavior is writing `pid -> expected_hash` mappings into a BPF map via LSM hook, but this is currently marked `TODO` and not wired.
- The interface is defined as `pin_allowed_execution(pid, expected_hash, domain)`.
- Operational syscall interception requires Linux and `CAP_BPF`.

Cerberus SASE

Source: `src/cerberus/sase/`

SubstrateManager Topologies

- `SINGLE_NODE`
- `HA_CLUSTER`
- `GEO_DISTRIBUTED`
- `EDGE`

ContainmentOrchestrator Controls

`ActionValidator` accepts only the following action types:

- `monitor`
- `alert`
- `block`
- `quarantine`
- `isolate`
- `redirect`
- `throttle`
- `log`
- `notify`

`MerkleProofGenerator` produces SHA-256 inclusion proofs for action/policy pairs.

`ContainmentRequest.to_hash()` computes a SHA-256 canonical request hash.

Observability

Source: `src/psia/observability/`

AutoimmuneDampener

Sensitivity tuning behavior:

- target false-positive rate: 5%
- sensitivity step: +/- 0.1 per cooldown period
- cooldown window: 10 decisions
- sensitivity range: 0.1 to 1.0
- `should_apply_rule(rule_id, score)` returns `score * sensitivity >= 0.5`

FailureDetector

Circuit-breaker and cascade logic:

- states: CLOSED, OPEN, HALF_OPEN
- 50% failure rate opens the circuit
- 2 or more open components trigger CascadeAlert
- 60-second recovery timeout moves to HALF_OPEN

Threat Model

Source: `src/psia/threat_model.py`

Threat classes:

- CRASH_FAULT
- BYZANTINE_FAULT
- COLLUSION
- VETO_ABUSE
- REPLAY_ATTACK

Resilience profiles:

Policy	N	Crash-safe	Byz-safe	Collusion-safe	Risk
unanimous	3	0	0	0	MEDIUM
2of3	3	1	0	0	HIGH
bft	4	1	1	1	LOW
bft	7	2	2	2	LOW

CollusionDetector uses a 100-vote sliding window, alerts when pair agreement exceeds 95%, and detects veto abuse when a head denies more than twice the global deny rate.

Documentation Corrections

The following repository pages should be updated to reflect verified implementation state.

Page	Issue	Correction
<code>cerberus/nir1.html</code>	Wrong expansion in title and code frame filename	UseNative Immune Reflex Layer and add Heart, MiniBrain, Antibody, Forge breakdown
<code>cerberus/waterfall.html</code>	Stage descriptions need correction	Confirm all 7 stages; abort decisions are deny and quarantine only
<code>cerberus/psia.html</code>	Invariant count incorrect	Correct to 9 root invariants

Page	Issue	Correction
codexdeus/shadow-thirsty.html	Filename uses thirsty not thirst	Retain filename as legacy; content is already corrected
All pages	Visual theme mismatch	Replace neon-tech canvas animations with deep-space starfields, planets, and nebulae

Assessment

This report reads as a canonical, code-faithful repository findings artifact suitable for architecture review, implementation documentation, governance traceability, and publication support. Its strongest quality is precision: enum values, control flow, severity mapping, state machines, signature mechanisms, and decision vocabularies are all captured at implementation granularity rather than described at a marketing or aspirational level.